

RoV Analytics: Where the Project Stands

26 September 2026

One paragraph

You are building a tool that turns screenshots of the RPL broadcast draft screen into team draft statistics for Arena of Valor (RoV). It has two halves. The first half, the **extractor**, is finished and running: a local Python web app on your PC where you attach screenshots, confirm the recognised draft, and save each game. Everything saved goes to your own Convex server on the NAS. The second half, the **dashboard**, is not started. It will read from that same Convex server and show the statistics: first picks, most banned heroes, response picks, side behaviour.

Where you are on the map

Piece	State
Data model (series, games, draft actions)	Done
Extractor web app	Done, redesigned, phone layout, LAN access
Image recognition of the draft bar	Working, improves as you confirm games
Hero reference table with art	Done, 131 heroes, 3 without art
Convex backend (self-hosted on NAS)	Done: games, draft actions, images, heroes
Real match data entered	Almost none: one series drafted, zero games saved
Dashboard	Not started
Git commit of all of this	Not done. Everything is uncommitted

What to do next, in order

1. Commit the work to git. It is a week of changes sitting uncommitted.
2. Enter real games. Confirm and save game 1 of FS vs BRU, then keep going. The dashboard is pointless without data, and recognition gets better with every saved game.

3. Decide the dashboard shape (questions in the last section), then build it.

Contents

Why this exists

The two programs

How the pieces connect

What was built this week

Current data

Next steps

Decisions that need you

Why this exists

The goal is draft statistics for your team, Full Sense (FS), in the Thai pro league RPL. The questions you want answered are things like: what does FS first pick, what gets banned against them, when the opponent picks hero X what does FS answer with, and how does side choice (blue or red) change behaviour across a series.

Public sites such as Liquipedia list picks and bans per game but not the interleaved order of the draft, and scraping them felt legally risky. So the source of truth is the broadcast itself: you take a screenshot of the draft bar at the end of the draft, and a second one of the post-game screen, and the tool reads them.

The project started in mid September 2026 as a minimap position tracker. On 24 September you decided to throw that away and

restart around the draft. The old tracker only exists in git history now.

Facts the design rests on

- RPL 2025-26: eight teams, double round robin, best of five. Playoffs best of seven.
- Global Ban-Pick rule: a team cannot pick a hero it already used earlier in the same series. Game 7 of a Bo7 resets this.
- The loser of the previous game picks side.
- The draft is 18 steps: 8 bans and 10 picks, alternating between sides in a fixed pattern that the tool knows for the RPL 2026 broadcast layout.
- The broadcast swaps hero-to-player mapping during the swap phase, so the final post-swap frame is the one to use for who played what.
- Flowborn is treated as separate heroes per form (Flowborn Carry, Flowborn Mage), decided 26 September. No combining.

The two programs

1. The extractor (built)

A Python program that runs on your PC. It serves a small web app in your browser. You open it by double-clicking `RoV_Extractor.cmd` in the repo root. That launcher installs dependencies if needed,

starts the server with auto-reload and LAN access, and opens the browser.

Inside the app there are four numbered stages for each game:

1. Pick or create a series (date, teams, best of, patch).
2. Attach the draft screenshot and the post-game screenshot. They are copied locally and also uploaded to Convex file storage, so they no longer need to live on the PC.
3. Recognise. The tool proposes all 18 draft actions, player names, K/D/A and the winner. Confident matches are marked, guesses are flagged. You fix anything wrong with dropdowns that show hero art.
4. Check and save. The checks column validates duplicates, Global Ban-Pick, and missing fields. Save writes the game locally and pushes it to Convex.

There is also a **Heroes page** (the /heroes route). It is the reference table of all 131 heroes with their ban icon and pick art. You verify each pairing and mark it Checked, by clicking the tick or by double-clicking (double-tapping on the phone) anywhere on the row. Saving the page syncs the hero list and art to Convex.

The app works on your phone over the home WiFi. The launcher prints the LAN address. There is no login, so it is for the home network only.

2. The dashboard (not built)

A web app that reads the data from Convex and shows the statistics. The agreed stack is TanStack Start (React) with Tailwind, living in a `web/` folder in this same repository, deployed to Vercel or Cloudflare with the root directory set to `web`. It talks to Convex directly; Python is not involved.

Nothing exists for it yet except the Convex tables it will read and a static mock-up at `docs/mockup/index.html`.

How the pieces connect

Read this top to bottom. Each arrow is one hop.

Screenshot on your PC or phone

|

v

Extractor web app (Python FastAPI, port 8787)

| OpenCV + RapidOCR read the draft bar

| You confirm the 18 actions

v

Local files: `data/series/<date>_<A>-<B>/gN.json`

|

v "Save" or `rov-extract push`

Convex on the NAS (<http://suwinas.local:3810>)

tables: `series`, `games`, `draftActions`, `heroes`, `images`

|

v (future)

Dashboard (TanStack Start) reads Convex, shows stats

Why Python and TypeScript together

Python does the image work because OpenCV and RapidOCR are Python libraries. Convex functions must be written in TypeScript, so the `convex/` folder holds the schema and mutations. Python calls those functions by name over HTTP using the Convex Python client. The dashboard will be TypeScript because Convex has first-class React bindings. Each language does the thing it is best at and they meet at the Convex API.

Where each thing lives

Path	What
RoV_Extractor.cmd	Double-click launcher
src/rov_extractor/	Python app: app.py (routes), recognise.py (vision), push.py (Convex), refdata.py (heroes and art), validate.py, cli.py
src/rov_extractor/static/	The web UI: index.html, app.js, style.css, and heroes.html/js/css
convex/	TypeScript backend: schema.ts, games.ts, files.ts, heroes.ts
data/ref/heroes.json	The 131 hero ids and names
data/ref/art-map.json	Hero to art-file pairing, Checked flags, Convex storage ids
data/ref/art/ data/ref/crops/	Seed art and learned crops (gitignored)
data/series/	Saved games and drafts
.env.local	Convex URL and admin key. Never commit, never share
docs/	Spec PDF, mock-up, architecture explainer

Commands

```
uv run rov-extract serve --open --lan --reload
uv run rov-extract list
uv run rov-extract validate
uv run rov-extract push                # all series
uv run rov-extract push --heroes       # hero table and art
uv run rov-extract purge-images        # delete local copies
already in Convex
bunx convex deploy --yes                # after editing convex/*.ts
```

What was built this week

24 September

- Removed the minimap tracker. Agreed the draft data model: a series, its games, and one row per draft action (sequence, side, team, ban or pick, hero, player).
- Built the extractor: manual slot-by-slot entry that mirrors the broadcast bar, autosave drafts, series folder layout, validation.
- Added recognition: OCR for names, scores, timer, result; hero identity from art; mapping post-game portraits back to draft slots.

25 September

- Heroes page: the reference table with ban icon, pick art, reassignment dropdowns, Checked flag, learned crops per hero.
- Full UI redesign after you said the first version looked like design slop. Near-black surface, one type family, three-column

layout, numbered stages, hero art everywhere, deep links per game.

- Convex self-hosted on the NAS connected. Backend written and deployed. Screenshots upload to Convex file storage on attach and can be restored if deleted locally.
- LAN mode so the phone can open the app. Phone layouts for both pages.

26 September

- Architecture explainer written and published to Postplan.
- Double-click or double-tap anywhere on a hero row toggles Checked. This took several attempts. The real cause was an orphaned old server process holding port 8787 on 127.0.0.1 while the launcher listened on 0.0.0.0, so your browser kept hitting stale code. Killed it. Lesson: if a change seems not to apply, check for a second server.
- Auto-reload for the Python server on any change under `src/`.
- Hero table and all hero art uploaded to Convex, so the dashboard can show hero pictures without touching the PC.
- Flowborn split into Flowborn Carry and Flowborn Mage as separate heroes. Forms column removed.

Current data

Heroes

Item	Value
Heroes in the table	131
With both ban icon and pick art	128
Missing art	Arthur, Brunhilda, Peura
Synced to Convex	Yes, 131 rows, art in storage

The Checked flags are yours to work through. Unchecked rows are the ones you have not verified yet. The Heroes page has an "unchecked only" checkbox to filter to them.

Games

One series exists: FS vs BRU on 20 September 2026, regular season, best of five, patch 1.60.1. Game 1 has both screenshots attached and a recognition proposal, but it has not been confirmed and saved. No game is in Convex yet. That means there are no statistics to show at all so far.

Convex

Instance `convex_rov` on the NAS at port 3810. Tables: series, games, draftActions, heroes, images. You said you will migrate this to a public Convex deployment yourself when it is time to publish. The

dashboard should therefore read the Convex URL from an environment variable so the switch is one line.

Next steps

Tick these off as you go.

Now

- ☐ Commit everything to git. Nothing from this week is committed. Check that `.env.local` is ignored before committing.
- ☐ Confirm and save FS vs BRU game 1 in the extractor. Then game 2 and onward.
- ☐ Work through the Heroes page until every row is Checked. Do it on the phone if that is more comfortable, double-tap to toggle.
- ☐ Find art for Arthur, Brunhilda and Peura, or let the learned crops cover them once they appear in a saved game.

Soon

- ☐ Enter five to ten more FS series. Recognition confidence should rise visibly as learned crops accumulate.
- ☐ Optional extractor improvement asked for but not built: a filter bar on the Heroes page (All, Unchecked, Checked, Missing art, with counts) instead of the single checkbox.

Then: the dashboard

- ☐ Answer the questions in the next section.
- ☐ Scaffold `web/` with TanStack Start, Tailwind, and the Convex React client.
- ☐ First screens: series list, game detail with the 18-step draft, hero table with first-pick rate, ban rate, pick rate, win rate.
- ☐ Response picks: given hero X picked by the opponent, what FS picked next.
- ☐ Side and series behaviour: blue vs red, game 1 vs later games.

Before publishing publicly

- ☐ Decide about the hero art. It is Garena artwork taken from a community broadcast package. Fine at home, questionable on a public site. Options: keep it private, ask permission, or replace with your own icons.
- ☐ Migrate Convex from the NAS to a public deployment (your job, you said).
- ☐ Do not expose the extractor. It has no login. Only the dashboard goes public.

Decisions that need you

These cannot be guessed from the code. Answering them unblocks the dashboard.

1. **Who is the dashboard for?** Only you and the FS coaching staff, or public? This decides whether it needs a login and whether the art question matters.
2. **Which statistics first?** Suggested order: hero table (picks, bans, wins), first-pick and first-ban, response picks, side split. Tell me if something else is more urgent, for example an opponent scouting view.
3. **Filters.** By tournament and stage? By patch? By opponent? By date range? Pick the two or three that you will use every time.
4. **Hosting.** Vercel or Cloudflare? Both work with a subdirectory. Cloudflare is cheaper at scale, Vercel is simpler to set up.
5. **Data entry pace.** Will you enter all of the current season, or only from now on? This decides whether backfilling from VODs is worth automating.
6. **Phone use of the extractor.** Is attaching screenshots from the phone the main path, or a nice-to-have? If main, the upload flow deserves more polish.

Shortest path to something useful: commit, save ten games, then build a single dashboard page with the hero table. Everything else can follow.